DevOps

CT2 PREP GUIDE

Unit 3: Infrastructure as Code (Terraform, Ansible)
Unit 4: Docker and Kubernetes

Exam	CT2 - 29.10.2026
Syllabus	Unit 3 and Unit 4
Pattern	10 MCQs + 15-mark Either/Or scenario-based question
Built from	Your Devops_u3.pdf and Devops_U4.pdf slides, arranged basic -> advanced, with syllabus gaps filled in

How to use this guide

- **Basic -> advanced:** every topic starts with a definition, then diagram, then comparison tables, then commands/code.
- **Diagrams:** practise redrawing Fig 3.1-3.7 and 4.1-4.10 - a clean labelled diagram is worth 3-4 marks of a 15-mark answer.
- **Red boxes** = syllabus topics that your PPT covers lightly or not at all (Terraform data sources, Ansible modules/plugins/collections detail, Docker Swarm).
- **Green boxes** = last-minute key points; **yellow boxes** = exam tips.
- **Section 3.11 and 4.8** contain Either/Or scenario questions with model answer outlines - write at least two of them fully on paper.

Contents

EXAM PATTERN AND ANSWER STRATEGY 3

A. The two parts of CT2 . . . 3
B. 15-mark answer template (about 2.5 - 3 pages) . . . 3
C. Source coverage - what comes from where . . . 3
D. Suggested study plan (28 Oct night + exam morning) . . . 3

UNIT 3 - INFRASTRUCTURE AS CODE (IaC) 4

3.1 Infrastructure as Code - Concepts . . . 4
3.2 Terraform - Overview . . . 6
3.3 Working with Data in Terraform - Input Variable Syntax . . . 7
3.4 Ansible - Introduction and Need . . . 9
3.5 Building Ansible Inventories . . . 11
3.6 Ansible Playbooks (needed for every scenario answer) . . . 11
3.7 Ansible Modules . . . 12
3.8 Working with Plugins . . . 13
3.9 Ansible Collections and Using Collections in Playbooks . . . 13
3.10 Terraform vs Ansible (compare question - 15M favourite) . . . 15
3.11 Scenario Questions with Model Answers (15 marks each) . . . 15
3.12 MCQ Practice - Unit 3 (30 questions) . . . 16

UNIT 4 - DOCKER AND KUBERNETES 19

4.1 Docker Overview . . . 19
4.2 Docker Architecture . . . 20
4.3 Docker Images and Containers . . . 20
4.4 Docker Swarm . . . 22
4.5 Kubernetes Overview . . . 23
4.6 Kubernetes Cluster Architecture . . . 24
4.7 Design of Pods . . . 25
4.8 Scenario Questions with Model Answers (15 marks each) . . . 29
4.9 MCQ Practice - Unit 4 (35 questions) . . . 30

ONE-PAGE QUICK REVISION SHEETS 32

Unit 3 - rapid recall . . . 32
Unit 4 - rapid recall . . . 32
Most-likely 15-mark topics (ranked) . . . 32

EXAM PATTERN AND ANSWER STRATEGY

A. The two parts of CT2

Part	Format	How to prepare
Part A	10 MCQs (Units 3 + 4)	Learn one-line definitions, command names, file names, defaults (bridge network, ClusterIP, RollingUpdate), and "which component does what". Use the quick-revision tables and MCQ banks.
Part B	15 marks, Either/Or, scenario-based	You get a business problem and must apply a tool. Practise a template: intro, diagram, steps, code/commands, benefits, conclusion.

B. 15-mark answer template (about 2.5 - 3 pages)

Section	Marks (suggested)	What to write
1. Introduction / definition	2	2-3 lines: define the technology and state the problem in the scenario.
2. Labelled diagram	3 - 4	Neat block diagram with arrows and 5-8 labels; add a one-line caption.
3. Components / working steps	5 - 6	Numbered steps or a short table; link each step to the scenario.
4. Code / commands / YAML	2	Only the essential part: 6-10 lines of config or 4-6 commands.
5. Benefits and conclusion	1 - 2	Three benefits tied to the scenario + one-line conclusion.

EXAM TIP

- Choose the Either/Or option for which you can **draw the diagram from memory**.
- Underline keywords (declarative, idempotent, agentless, desired state, self-healing, control plane) - examiners scan for them.
- Never leave the diagram unlabelled; never write only a definition for a scenario question.

C. Source coverage - what comes from where

Topic (syllabus)	Your PPT	This guide
IaC concepts, push/pull, benefits, plugin architecture	Covered (slides + quiz)	Expanded + declarative/mutable comparison
Terraform overview, HCL, init/plan/apply/destroy/validate, EC2 example	Covered	Expanded with architecture and workflow diagrams
Input variable syntax, output variables	Covered (types, outputs)	Added tfvars, precedence, data sources, locals (red box)
Ansible intro, need, concepts, editions, inventory, playbooks, EE	Covered	Expanded with examples and architecture diagram
Ansible modules, plugins, collections, using collections in playbooks	Definitions only	Full sections + FQCN + install + playbook usage (red boxes)
Docker overview, architecture, images, containers, storage, network, CLI	Covered	Expanded with diagrams, Dockerfile, run flow
Docker Swarm	Not covered (1 mention)	Full section 4.4 (red box)
Kubernetes overview, architecture, objects, networking, kubectl	Covered (large deck)	Condensed to exam-relevant parts + diagrams
Design of pods	Pod YAML + networking rules	Patterns, probes, resources, multi-container YAML

D. Suggested study plan (28 Oct night + exam morning)

- **Block 1 (Unit 3 theory):** sections 3.1 - 3.3; redraw Fig 3.1, 3.2, 3.3, 3.4; write Terraform main.tf from memory.
- **Block 2 (Ansible):** sections 3.4 - 3.10; write inventory + playbook from memory; learn module vs plugin vs collection.
- **Block 3 (Docker + Swarm):** sections 4.1 - 4.4; redraw Fig 4.1, 4.2, 4.3, 4.5; memorise the CLI table and network drivers.

- **Block 4 (Kubernetes):** sections 4.5 - 4.7; redraw Fig 4.6, 4.7, 4.8, 4.9; write pod and deployment YAML.
- **Block 5:** do both MCQ banks; then the one-page revision sheet at the end; skim scenario answers once more in the morning.

UNIT 3 - INFRASTRUCTURE AS CODE (IaC)

Syllabus: IaC concepts | Terraform overview | Working with data in Terraform (input variable syntax) | Introduction to Ansible, need of Ansible, building Ansible inventories | Ansible modules | Working with plugins | Ansible collections | Using collections in playbooks.

How this unit maps to the exam

- 10 MCQs: definitions, commands, file names, "which tool does what", push vs pull, idempotency, FQCN.
- 15-mark scenario: usually "A company has problem X -> propose IaC / Terraform / Ansible solution with diagram, files, commands, benefits".
- Red boxes in this guide mark topics that are in the syllabus but thin or missing in your PPT - revise them separately.

3.1 Infrastructure as Code - Concepts

A. Definition (write this first in any answer)

Infrastructure as Code (IaC) is the practice of managing and provisioning IT infrastructure (servers, networks, load balancers, databases, storage) through machine-readable definition files (code) instead of manual configuration or interactive tools. The code is stored in version control, reviewed, tested and executed by automation tools.

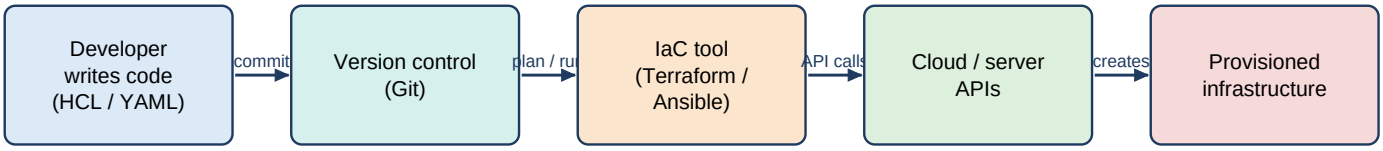


Fig 3.1 Generic Infrastructure-as-Code workflow

B. Why IaC? (problem it solves)

Before automation, DevOps/Ops engineers manually did: setup servers, configure network, create route tables, install apps, maintain and update them (your PPT "DevOps tasks before automation"). Manual work is slow, error-prone and not repeatable, leading to **configuration drift** - dev, test and production slowly become different.

Benefit	Explanation (use as a 2-mark point)
Scalability	Resources can be scaled up/down quickly by changing a number in code; efficient allocation and cost saving.
Consistency	Same code = same environment every time; removes drift and "works on my machine" problems.
Version control	Infra changes are tracked in Git: history, audit trail, review, rollback to a previous state.
Automation	Provisioning, configuration and orchestration run automatically - faster deployments, fewer human errors.
Collaboration	Teams share, review (pull requests) and reuse infra code; improves communication.
Compliance / security	Standard configs and security policies are enforced and auditable.

C. Three main task categories (from your PPT)

Category	What it covers	Typical tool
1. Infrastructure provisioning	Spinning up servers, network configuration, creating load balancers	Terraform (also CloudFormation)
2. Configuration of provisioned infrastructure	Installing apps on servers, managing them, preparing servers to deploy apps	Ansible, Chef, Puppet
3. Deployment of application	Pulling images / code and running the app (e.g. Docker containers)	Ansible, CI/CD, Kubernetes

D. Distinction of phases

Initial setup phase: provision infrastructure, configure it, initial installation and configuration of software. **Maintaining phase:** adjust infrastructure, add/remove servers, update software, re-configure software.

E. Push vs Pull approach

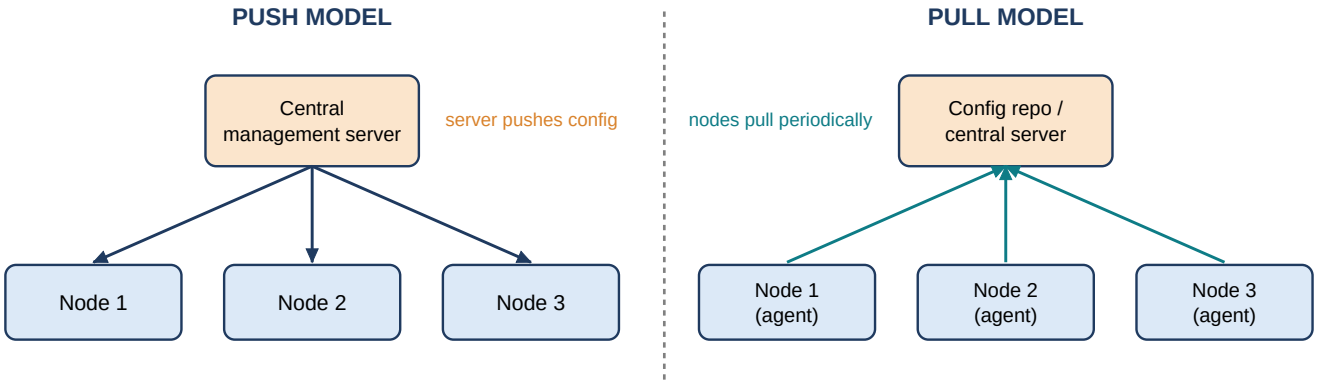


Fig 3.2 Push vs Pull deployment in IaC (Ansible = push, Puppet/Chef agents = pull)

Point	Push model	Pull model
Who starts?	Central management server initiates and controls deployment	Nodes periodically fetch config from central repo/server
Control	Centralised	Decentralised - nodes decide when to update
Agent on node?	Not required (Ansible uses SSH)	Usually agent required (Puppet, Chef)
Example tools	Ansible, Terraform (CLI-driven)	Puppet, Chef, GitOps agents

F. How IaC tools work: declarative vs procedural, mutable vs immutable (from your PPT)

Aspect	Declarative	Procedural / Imperative
Idea	Declare the end result ("I want 2 servers")	Give step-by-step instructions ("create a server, add a server, make this change")
Re-running	Tool computes the difference - safe	May create duplicates unless scripted carefully
Disadvantage	-	Limited expressiveness; hard to see the desired state (slide quiz Q9)
Tools	Terraform, CloudFormation, (Ansible modules are declarative in nature)	Shell scripts, Chef recipes

Aspect	Mutable infrastructure	Immutable infrastructure
Meaning	Changes are made in place on existing servers	Servers are never modified - they are replaced with new ones
Slide keyword	Changes	Replacing
Risk / benefit	Drift builds up over time	Predictable, easy rollback (Terraform + images/containers)

G. Plugin-based architecture in IaC

- Core tool stays small; functionality is added through **plugins** (e.g. Terraform providers, Ansible connection/filter/callback plugins).
- **Features:** flexibility, modularity, scalability, extensibility (add features without rebuilding the system).
- **Advantages:** flexibility, scalability (add/remove plugins), reusability across projects.
- **Challenges:** complex integration, version control of many plugins, security of the plugin ecosystem.
- **Best practices:** modularity, standardised interfaces, version control for plugins.

H. Which tool for which job? (slides "Difference of IaC tools")

Tool	Strength	Initial infra setup	Manage infra	Initial app setup	Manage apps
Terraform	Provisioning (declarative, multi-cloud)	YES	YES	limited	limited
Ansible	Configuration management + deployment	NO (slide: red X)	YES	YES	YES
Chef / Puppet	Configuration management (agent-based)	NO	YES	YES	YES

KEY POINTS TO REMEMBER

- Best combination: **Terraform to provision (and manage) infrastructure, Ansible to install and deploy applications**.
- No single tool does everything - that is why there are many tools.
- IaC works on both public and private clouds (slide quiz Q10).

3.2 Terraform - Overview

Terraform is an open-source IaC tool developed by **HashiCorp**. It lets you describe and provision infrastructure in a **declarative** way using **HCL (HashiCorp Configuration Language)**: you declare the desired state and Terraform works out the steps. It creates and manages resources on cloud platforms through their **APIs**; **providers** allow it to work with any platform that has an accessible API.

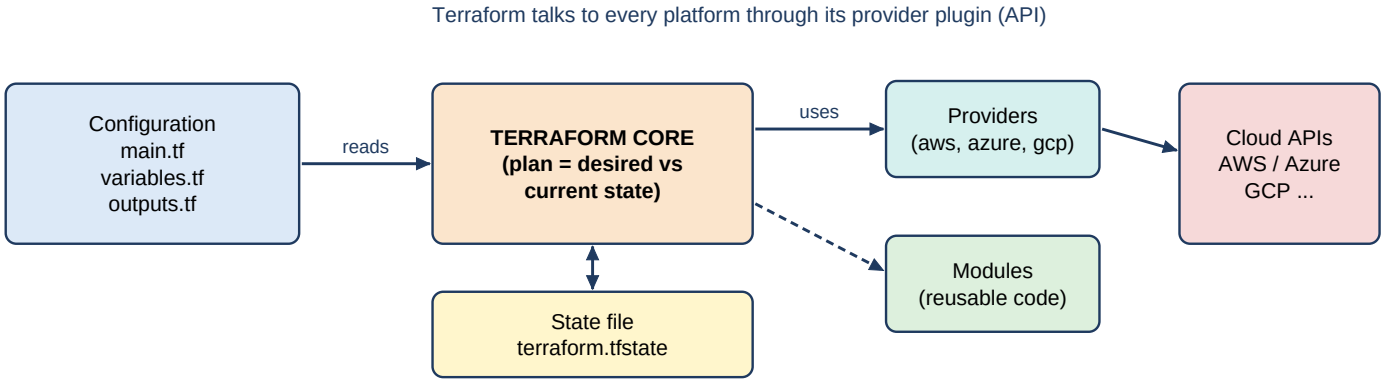


Fig 3.3 Terraform architecture

Key feature	Explanation
Declarative configuration	Describe what you want, not how to get it.
Multi-cloud support	Cloud-agnostic: AWS, Azure, Google Cloud, etc. with a single tool.
Resource provisioning	VMs, networks, databases defined in scripts written in HCL.
State management	State file records the current infrastructure; used to plan and apply only the needed changes.
Modularity and reusability	Configurations broken into reusable modules - standardised components.

HCL - features (as in your PPT)

- **Declarative syntax** - "what", not "how".
- **Blocks and nesting** - file is made of blocks (resource, provider, variable, output); blocks can be nested to show dependencies.
- **Attributes** - settings inside blocks (ami, instance_type).
- **Variables** - parameterise and reuse values.
- **Expressions** - calculations, string manipulation, conditions.
- **Comments** - # single-line and `/* */` multi-line.

Typical project files

File	Purpose
main.tf	Main configuration: provider + resources
variables.tf	Input variable declarations
outputs.tf	Output values to display after apply
terraform.tfvars	Values for the input variables
terraform.tfstate	State file - current known infrastructure (never edit by hand; use remote backend + locking for teams)
.terraform/	Downloaded provider plugins and modules (created by init)

Terraform commands (very high probability MCQ + 15M)



Fig 3.4 Terraform core workflow (write -> init -> validate -> plan -> apply -> destroy)

Command	What it does (PPT)
terraform init	Initialises the working directory: reads main.tf/variables.tf/outputs.tf, initialises the backend, downloads provider plugins from the Terraform Registry, initialises modules, sets up local state.
terraform validate	Checks syntax and validity of configuration files. Prints nothing/"Success" if correct; otherwise shows errors (e.g. Invalid character on main.tf line 5).
terraform plan	Previews changes without applying. Classifies into **Additions** (to create), **Modifications** (to update), **Deletions** (to destroy).
terraform apply	Executes the planned changes; asks you to type **yes** to confirm.
terraform destroy	Destroys all resources created by the configuration; also asks for confirmation.
terraform fmt / show / output	Format code / show state or plan / print output values (extra, good to mention).

Worked example - create an EC2 instance (from your PPT)

Steps: 1) Install Terraform 2) Configure AWS credentials 3) Create configuration file 4) terraform init 5) terraform plan 6) terraform apply (type yes) 7) terraform destroy (type yes).

```
# main.tf
provider "aws" {
  region = "us-west-2"                # AWS region
}

resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"   # Amazon Linux 2 AMI ID
  instance_type = "t2.micro"

  tags = {
    Name = "example-instance"
  }
}
```

- `provider "aws"` block -> which plugin/cloud and region to use.
- `resource "aws_instance" "example"` -> resource type + local name; reference it later as aws_instance.example.
- Arguments (ami, instance_type, tags) = attributes of the resource.

3.3 Working with Data in Terraform - Input Variable Syntax

"Data" in Terraform = values that flow **into** the configuration (input variables), **out of** it (output variables) and values **looked up** from existing infrastructure (data sources) or computed inside it (locals). Your PPT covers input and output variables.

A. Input variables - syntax

```
variable "<NAME>" {
  description = "what this variable is for"   # optional
  type        = string                         # string | number | bool | list(...) | map(...) | set | object
  default     = "t2.micro"                     # optional; if missing, Terraform asks for a value
  sensitive   = false                          # true hides value in output
}

# using it inside configuration:   instance_type = var.<NAME>
```

Primitive and collection types (PPT examples):

```
variable "example_string" {
  type    = string
  default = "Hello, Terraform!"
}

variable "example_boolean" {
  type    = bool
  default = true
}
```

```
variable "example_number" {
  type    = number
  default = 42
}

variable "example_list" {
  type    = list(string)
  default = ["item1", "item2", "item3"]
}
```

Type	Meaning	Example value
string	Sequence of characters in double quotes	"us-west-2"
number	Integer or floating-point	42, 3.14
bool	true / false	true
list(type)	Ordered collection	["a","b","c"] -> var.x[0]
map(type)	Key-value pairs	{ env = "dev", team = "ops" } -> var.x["env"]
set(type) / object({...}) / tuple	Unique unordered values / structured record / fixed mixed list	(advanced)

B. Ways to supply values (and precedence)

- **default** value in the variable block (lowest).
- **Environment variable** `TF_VAR_instance_type=t2.small`.
- **terraform.tfvars** / ***.auto.tfvars** files (e.g. `instance_type = "t2.small"`).
- **Command line** `terraform apply -var="instance_type=t2.large"` or `-var-file=prod.tfvars` (highest).
- If nothing is supplied and there is no default, Terraform prompts interactively.

Order from low to high priority: default -> env var (TF_VAR_) -> terraform.tfvars -> *.auto.tfvars -> -var / -var-file.

C. Output variables (PPT slide)

```
output "public_ip" {                    # single value
  value = aws_instance.example.public_ip
}

output "instance_info" {                # multiple values as a map
  value = {
    id         = aws_instance.example.id
    public_ip  = aws_instance.example.public_ip
    private_ip = aws_instance.example.private_ip
  }
}
```

- **Single value** (IP address, instance ID), **multiple values** (map/list) and **complex outputs** (nested maps/lists, e.g. subnet_ids and cidr_blocks).
- Printed after `terraform apply` or any time with `terraform output`; also used to pass values between modules.

NOT IN YOUR PPT - ADD FROM SYLLABUS

- **Data sources** read existing information: `data "aws_ami" "latest" { most_recent = true owners = ["amazon"] }` -> used as `data.aws_ami.latest.id`. Good for "do not hard-code the AMI".
- **Locals** are named expressions inside a module: `locals { name = "web-${var.env}" }` -> `local.name`.
- **Variable validation:** `validation { condition = length(var.name) > 3 error_message = "..." }`.
- **Meta-arguments:** count and for_each create many resources from one block; depends_on forces ordering.

D. Full example with variables and outputs

```
# variables.tf
variable "region" {
  type    = string
  default = "us-west-2"
}
variable "instance_type" {
  type    = string
  default = "t2.micro"
}

# main.tf
provider "aws" {
  region = var.region
}
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_type
  tags = {
    Name = "web-server"
  }
}

# outputs.tf
output "web_public_ip" {
  value = aws_instance.web.public_ip
}

# run:   terraform init -> validate -> plan -> apply -var="instance_type=t2.small"
```

3.4 Ansible - Introduction and Need

Ansible is an open-source, command-line **IT automation** tool written in **Python** and developed by **Red Hat**. It is used to **configure systems, deploy software and orchestrate advanced workflows**; typical tasks are application deployment, system updates, configuration management. It uses a declarative language (YAML) to define the desired state.

Why do we need Ansible? (Need - 5 mark point)

- Manual configuration of tens/hundreds of servers is slow, inconsistent and error-prone.
- **Agentless:** nothing to install on managed nodes - low maintenance overhead (uses OpenSSH).
- **Simple:** human-readable YAML playbooks that "read like documentation"; minimal learning curve.
- **Idempotent:** running a playbook many times gives the same final state - safe re-runs.
- **Scalable and flexible:** modular design; supports many OSes, clouds and network devices.
- **Secure and reliable:** minimal moving parts, existing SSH credentials.
- **Use cases:** remove repetition, manage system configuration, continuously deploy complex software, zero-downtime rolling updates.

Ansible editions (PPT comparison)

	Community Ansible	AWX	Red Hat Ansible Automation Platform
Nature	Free, open-source command-line tool	Free, unsupported open-source GUI + API wrapped around Ansible	Subscription enterprise product combining 20+ projects
Support	Community	Not supported	At least 18 months enterprise support per release
Content	Community collections	Community	140+ certified collections, 60+ partners; trusted chain-of-custody
Extras	-	-	Automation analytics, Red Hat Insights, legal (IP) protection, supported upgrades/migrations

Ansible concepts (the vocabulary list - learn the one-line definitions)

Term	Definition
Control node	Machine where Ansible is installed and from which you run ansible, ansible-playbook, ansible-vault, ansible-inventory. Laptop, server or container.
Managed node (host)	Target device (server, network appliance) that Ansible manages. No agent needed.
Inventory	List of managed nodes, logically organised in groups, with info like IP address; used for node selection and bulk variable assignment.
Playbook	YAML file; basic unit of Ansible execution; automation blueprint that declares the desired state; list of plays.
Play	Ordered list of tasks mapped to managed nodes in the inventory.
Task	Reference to a single module with its arguments - the operation to perform.
Module	Unit of code Ansible copies to and executes on a managed node to do the task (apt, copy, service ...).
Plugin	Code that extends Ansible core: connection, filter, callback, lookup ... (runs on the control node).
Collection	Distribution format bundling playbooks, roles, modules and plugins; installed via Ansible Galaxy.
Role	Reusable structured grouping of tasks, handlers, variables, templates.
Handler	Special task that runs only when notified by another task changed something (e.g. restart service).

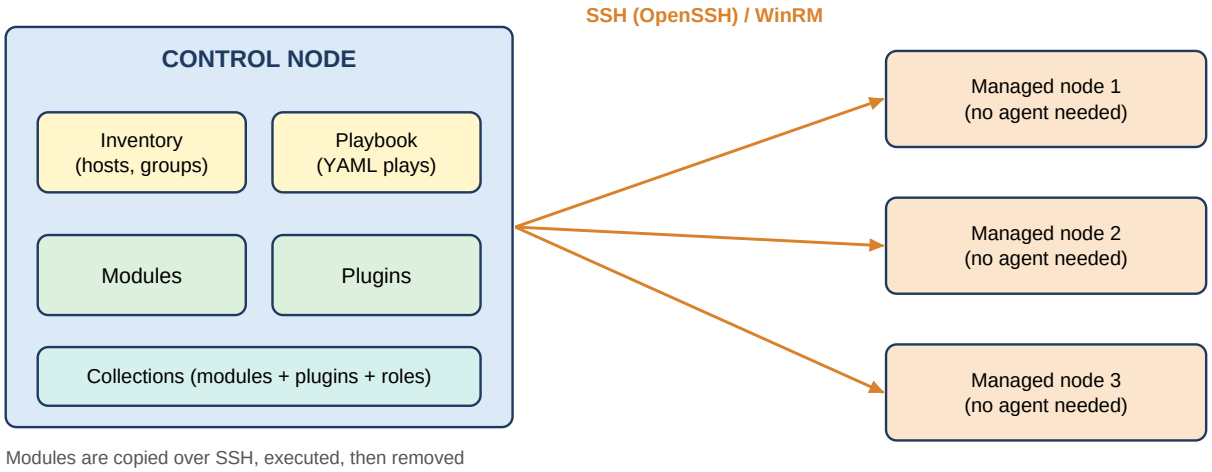


Fig 3.5 Ansible agentless architecture

Four Ansible principles (PPT)

- **Agent-less architecture** - no extra software on managed nodes.
- **Simplicity** - YAML playbooks; decentralised, uses existing SSH/OS credentials.
- **Scalability and flexibility** - modular design for many OSes, clouds, network devices.
- **Idempotence and predictability** - if the system is already in the described state Ansible changes nothing.

Other features (Role of Ansible slides): static or dynamic inventory; RBAC in Ansible Tower (web UI) so only authorised users run specific playbooks; extensible - custom modules, plugins and integrations.

3.5 Building Ansible Inventories

An inventory organises managed nodes in centralised files that give Ansible system information and network locations, so one command can manage a large number of hosts. It can be **static** (a file you write) or **dynamic** (generated from cloud APIs such as AWS/Azure). Formats: **INI** (simple, good for few nodes) or **YAML** (better as the number of nodes grows).

Steps to build an inventory (PPT)

- Run the hosts locally in a container or VM.
- Add your public SSH key to each host.
- Create a file with .ini extension (e.g. inventory.ini).
- Add a new group to the file and specify the IP addresses.
- Verify the inventory (ansible-inventory --list).
- Ping the group (ansible myhosts -m ping).

```
# inventory.ini  (INI format)
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1  ansible_host=192.168.1.20

# group of groups
[production:children]
webservers
dbservers

# group variables
[webservers:vars]
ansible_user=ubuntu
http_port=80

# inventory.yaml  (same inventory in YAML)
all:
  children:
    webservers:
      hosts:
        web1: { ansible_host: 192.168.1.10 }
        web2: { ansible_host: 192.168.1.11 }
      vars: { ansible_user: ubuntu }
    dbservers:
      hosts:
        db1: { ansible_host: 192.168.1.20 }
```

Command	Purpose
ansible-inventory -i inventory.ini --list	Show the parsed inventory (verify)
ansible all -i inventory.ini -m ping	Ad-hoc ping of all hosts (SSH + Python check)
ansible webservers -i inventory.ini -m apt -a "name=nginx state=present" --become	Ad-hoc command using a module
ansible-playbook -i inventory.ini site.yml	Run a playbook
ansible-playbook site.yml --check	Dry run - shows what would change

Implicit groups: `all` (every host) and `ungrouped`. **Variables:** host vars, group vars, and files in group_vars/ and host_vars/ directories.

3.6 Ansible Playbooks (needed for every scenario answer)

Playbooks are simple, human-readable YAML scripts that declare the desired state of local or remote systems; Ansible makes sure the system stays in that state. A playbook is a list of plays; a play maps hosts to ordered tasks; a task calls one module.

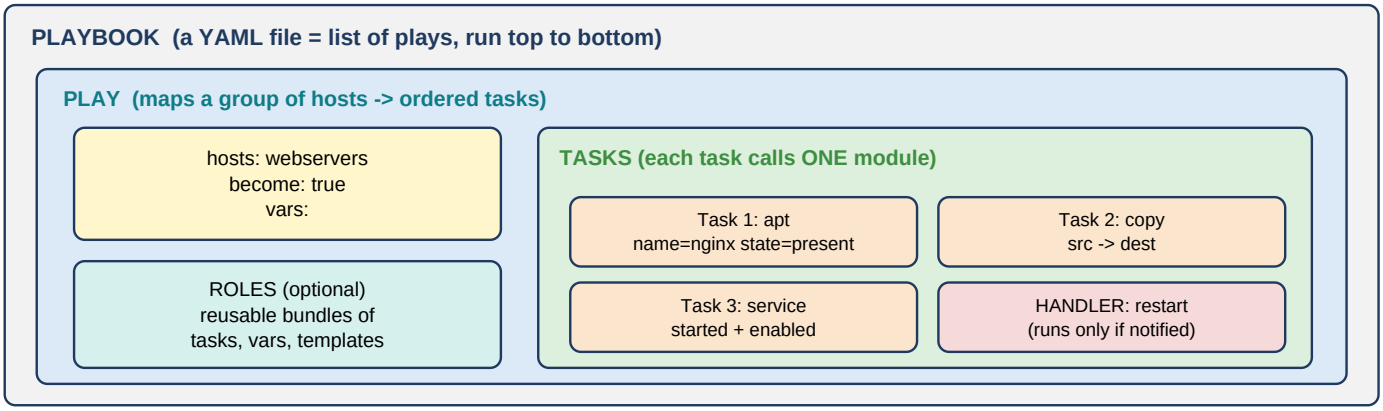


Fig 3.6 Playbook > Play > Task > Module hierarchy

```
---                                     # YAML document start
- name: Basic playbook                  # play name (optional)
  hosts: localhost                      # target hosts from inventory
  tasks:                                # the actual work
    - name: Print a message
      debug:
        msg: "Hello, World!"
```

*The PPT slide: the playbook has three sections - **name, hosts and tasks**; "---" is the YAML delimiter that starts the document.*

```
---
- name: Configure web servers
  hosts: webservers
  become: true                          # sudo
  vars:
    pkg: nginx
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: "{{ pkg }}"
        state: present
        update_cache: true
    - name: Deploy home page
      ansible.builtin.copy:
        src: index.html
        dest: /var/www/html/index.html
      notify: Restart nginx
    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

3.7 Ansible Modules

NOT IN YOUR PPT - ADD FROM SYLLABUS

- Your PPT gives only the definition and "built-in modules for package installation, file manipulation, service management". Learn the table below for the 15-mark answer.

Module = the code that Ansible copies to and executes on each managed node (when needed) to accomplish the action defined in a task. Modules are small programs (mostly Python), run in the order defined by the playbook, and return JSON (changed / ok / failed). One task invokes one module; a playbook can use many different modules. Modules are grouped into collections and named with an **FQCN** (Fully Qualified Collection Name), e.g. `ansible.builtin.copy`.

Category	Common modules	Example use
Package management	apt, yum / dnf, package, pip	Install nginx
Files	copy, file, template, lineinfile, fetch	Push config file, set permissions
Services	service, systemd	Start/enable/restart a service

Category	Common modules	Example use
Users and system	user, group, cron, reboot	Create user, schedule job
Commands	command, shell, script, raw	Run commands (not idempotent by default)
Utility	debug, ping, setup (facts), assert, wait_for	Print variables, gather host facts
Source / web	git, get_url, uri	Clone repo, call REST API
Cloud / network	amazon.aws.ec2_instance, azure.azcollection.*, cisco.ios.*	Provision VM / configure switch

- **Idempotent modules** (apt, copy, service) check current state and change only if needed; `command/shell` always run unless guarded with `creates:` / `when:`.
- Documentation: `ansible-doc ansible.builtin.copy`.
- Ad-hoc: `ansible all -m ping` (one module, no playbook).

3.8 Working with Plugins

Plugins are pieces of code that **expand Ansible's core capabilities**. They control how Ansible connects to a managed node (**connection plugins**), manipulates data (**filter plugins**) and even what is displayed in the console (**callback plugins**).

Plugin type	Role	Example
Connection	How to connect to managed node	ssh, local, winrm, docker
Filter	Transform data inside templates/playbooks	{{ name | upper }}, to_json, default()
Callback	Control/format console output and events	default, json, timer, profile_tasks
Lookup	Fetch data from outside sources	lookup('file','/etc/hosts'), env, password
Inventory	Build inventory dynamically	aws_ec2, azure_rm, yaml, ini
Become	Privilege escalation	sudo, su
Strategy	How tasks are scheduled	linear, free
Vars / Cache / Test	Extra variable sources, fact caching, tests used in "when"	redis cache, "is defined"

EXAM TIP

- **Module vs plugin (frequent 5-mark question):** modules run on the **managed node** to do a task; plugins run on the **control node** and extend Ansible itself.
- Plugins ship inside collections or can be custom-written in Python and placed in a plugin directory (e.g. filter_plugins/, lookup_plugins/ next to the playbook or in a role).
- Enable/choose a plugin in ansible.cfg, e.g. `stdout_callback = yaml`, `callbacks_enabled = timer`.

3.9 Ansible Collections and Using Collections in Playbooks

A **collection** is a format in which Ansible content is distributed. It can contain **playbooks, roles, modules and plugins**. You install and use collections through **Ansible Galaxy** (public hub) or Automation Hub (certified content). Name format: `namespace.collection` (e.g. amazon.aws, community.general, cisco.ios, ansible.builtin).

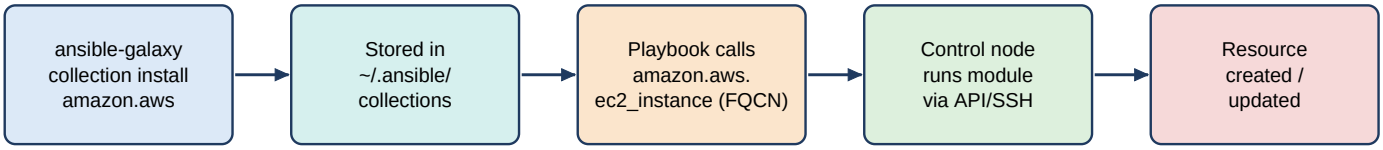


Fig 3.7 Using an Ansible collection in a playbook

Installing collections

```
ansible-galaxy collection install amazon.aws                # one collection
ansible-galaxy collection install community.general:8.0.0   # specific version
ansible-galaxy collection install -r requirements.yml       # many, from a file
ansible-galaxy collection list                               # what is installed

# requirements.yml
collections:
  - name: amazon.aws
  - name: community.general
    version: ">=8.0.0"
```

Using a collection inside a playbook

Method 1 - FQCN (recommended): write the full name in the task.

```
---
- name: Launch an EC2 instance
  hosts: localhost
  tasks:
    - name: Create instance
      amazon.aws.ec2_instance:           # <namespace>.<collection>.<module>
        name: demo-web
        instance_type: t2.micro
        image_id: ami-0c55b159cbfafe1f0
        region: us-west-2
        state: running
    - name: Show message
      ansible.builtin.debug:
        msg: "Done"
```

Method 2 - collections keyword: list collections at play level and use short module names (searched in the listed order).

```
- hosts: localhost
  collections:
    - amazon.aws
  tasks:
    - ec2_instance:                    # resolves to amazon.aws.ec2_instance
        name: demo-web
        ...
```

- Using FQCN avoids ambiguity when two collections have a module with the same name.
- Collections can be installed from tarballs, Git repos or private Automation Hub.
- **Execution Environments (EE)** - your PPT: Ansible control nodes packaged as **container images** so everyone runs the same Ansible + collections + Python packages. Tools: Ansible Builder, Navigator, AWX, Runner, VS Code Dev Containers.

Execution Environment - quick flow (PPT)

- Install docker/podman and python3; `pip3 install ansible-navigator`; verify with `ansible-navigator --version`.
- Create folder and `execution-environment.yml`; build: `ansible-builder build --tag postgresql_ee`; list: `podman image list`.
- Run playbook inside EE: `ansible-navigator run test_localhost.yml --execution-environment-image postgresql_ee --mode stdout`.
- For a remote host you need an IP/hostname, valid credentials and a sudo user; add `-i inventory -u student -k -K`.

3.10 Terraform vs Ansible (compare question - 15M favourite)

Parameter	Terraform	Ansible
Developer	HashiCorp	Red Hat
Primary purpose	Infrastructure provisioning (create/destroy resources)	Configuration management, application deployment, orchestration
Language	HCL	YAML (playbooks), Python (modules)
Approach	Declarative; keeps a **state file**	Mostly declarative via modules; procedural task order; **no state file**
Architecture	Client-only; talks to cloud APIs via providers	Agentless; control node -> SSH/WinRM -> managed nodes
Infrastructure style	Immutable (replace resources)	Mutable (change in place)
Core commands	init, validate, plan, apply, destroy	ansible, ansible-playbook, ansible-galaxy, ansible-inventory
Extensibility	Providers and modules	Modules, plugins, collections, roles
Best together	Provision servers/networks	Configure servers, install and deploy apps

3.11 Scenario Questions with Model Answers (15 marks each)

EXAM TIP

- Write every 15M answer in this order: **(1) Definition/Intro (2M) - (2) Diagram (4M) - (3) Components/working in steps (6M) - (4) Code/commands relevant to the scenario (2M) - (5) Benefits + conclusion (1M)**.
- Always name the scenario in your answer ("In this case the company needs ...") - scenario questions reward application, not only theory.

Q1 (Either) A start-up deploys servers manually; dev, test and production behave differently and releases fail. Propose an IaC solution.

- **Problem analysis:** manual setup -> human errors, configuration drift, slow scale-up, no audit trail.
- **IaC definition** and why it fits: infrastructure described in code, stored in Git, executed by tools (Fig 3.1).
- **Proposed design:** Terraform provisions servers/network/load balancer (declarative, state file); Ansible configures servers and deploys the application; Git + pull-request review; same code reused for dev/test/prod with different variable files.
- **Workflow:** code -> commit -> terraform plan (review) -> apply -> ansible-playbook -> verify. Push model: central controller pushes changes.
- **Benefits:** consistency, scalability, version control and rollback, automation, collaboration, compliance. **Challenges:** learning curve, state management, secret handling.
- **Conclusion:** IaC makes the environment reproducible and releases predictable.

Q1 (Or) Compare push and pull models and declarative vs procedural IaC, with examples.

- Define both, draw Fig 3.2, give the comparison table (who initiates, control, agent, tools).
- Declarative = desired end state (Terraform); procedural = step-by-step (scripts). Mention mutable vs immutable.
- Conclude which suits which scenario (large fleet with agents -> pull; ad-hoc orchestration -> push).

Q2 (Either) A team wants to launch an EC2 web server on AWS and print its IP. Show how Terraform does it, using variables and outputs.

- Intro to Terraform (HashiCorp, HCL, declarative, providers, state) + Fig 3.3 architecture.
- Files: main.tf (provider+resource), variables.tf (region, instance_type with types/defaults), outputs.tf (public_ip). Write the full code from section 3.3-D.
- Commands with purpose: init (download AWS provider) -> validate -> plan (1 to add) -> apply (type yes) -> terraform output -> destroy.
- Explain how variable values can be overridden: -var, terraform.tfvars, TF_VAR_ environment variable.

- State file role; benefits (repeatable, version-controlled); conclusion.

Q2 (Or) Explain the Terraform workflow and the commands init, plan, apply, destroy with the effect of each on the infrastructure.

- Draw Fig 3.4; explain additions / modifications / deletions in plan output; confirmation prompt (yes) in apply and destroy; state updates after each.

Q3 (Either) A company must install and configure Nginx on 50 Ubuntu servers and restart it only when the config changes. Design the Ansible solution.

- Intro: Ansible agentless automation tool; need (manual config is slow/inconsistent). Fig 3.5 architecture.
- **Step 1 Inventory** - group webservers (INI/YAML) with ansible_user; verify with ansible-inventory and ping.
- **Step 2 Playbook** - hosts: webservers, become: true; tasks use apt (install), copy/template (config), service (enable/start); **handler** "Restart nginx" notified by the config task (Fig 3.6 + code in 3.6).
- **Step 3 Run** - ansible-playbook -i inventory.ini site.yml (use --check first).
- **Idempotency:** second run shows ok, changed=0; handlers run only on change. Scales to all 50 hosts in parallel (forks).
- Benefits and conclusion.

Q3 (Or) Explain Ansible components - control node, managed node, inventory, playbook, play, task, module - and the four principles of Ansible.

- Use the concept table in 3.4, Fig 3.5/3.6; add agentless, simplicity, scalability, idempotence with one example each.

Q4 (Either) An organisation manages AWS instances and Cisco switches with Ansible. How do collections and plugins help? Show usage in a playbook.

- Define collection (modules + plugins + roles + playbooks, namespace.collection, Galaxy) and plugin (connection/filter/callback/lookup/inventory).
- Install: ansible-galaxy collection install amazon.aws cisco.ios (or requirements.yml). Fig 3.7.
- Playbook using FQCN: amazon.aws.ec2_instance for AWS; cisco.ios.ios_config for switches (connection plugin ansible.netcommon.network_cli).
- Dynamic inventory plugin aws_ec2 builds the host list from AWS; callback plugin for readable output; filter plugins for data.
- Benefits: modular distribution, versioning, no core upgrade needed, certified content; FQCN avoids name clashes.

Q4 (Or) Differentiate Terraform and Ansible and explain how they are used together in one pipeline.

- Comparison table 3.10 + pipeline: Terraform provisions VMs -> outputs IPs -> inventory (static/dynamic) -> Ansible installs apps -> app deployed (container). Mention "Terraform for provisioning, Ansible for configuration/deployment".

3.12 MCQ Practice - Unit 3 (30 questions)

Attempt without looking, then check the key at the end of this section. Each MCQ in CT2 is one-line recall - read the "Key points" boxes before attempting.

1. IaC stands for
(A) Internet as Code (B) Infrastructure as Code (C) Integration as Cloud (D) Instance as Config

2. Which of these is NOT a benefit of IaC?
(A) Version control (B) Consistency (C) Increased manual intervention (D) Scalability

3. In the push model of IaC
(A) Nodes pull from repo (B) A central server initiates and pushes changes (C) Only dev environment is updated (D) No server is used

4. Terraform was developed by
(A) Red Hat (B) Google (C) HashiCorp (D) Docker Inc

5. Terraform configuration language is
(A) YAML (B) JSON only (C) HCL (D) XML

6. Which command downloads provider plugins?
(A) terraform plan (B) terraform init (C) terraform apply (D) terraform fmt

7. terraform plan output classifies changes as
(A) Add, Modify, Delete (B) Start, Stop, Pause (C) Build, Ship, Run (D) Pull, Push, Merge

8. Which command removes all resources created by a configuration?
(A) terraform remove (B) terraform clean (C) terraform destroy (D) terraform reset

9. The file that records Terraform's current view of the infrastructure is
(A) main.tf (B) terraform.tfstate (C) outputs.tf (D) variables.tf

10. Which is a valid Terraform variable type?
(A) integer32 (B) list(string) (C) array (D) char

11. How do you refer to a variable named region?
(A) $region (B) var.region (C) @region (D) variable(region)

12. Which has the HIGHEST precedence for a variable value?
(A) default (B) TF_VAR_ env variable (C) terraform.tfvars (D) -var on command line

13. The Terraform block used to print values after apply is
(A) variable (B) output (C) provider (D) local

14. Ansible is written in
(A) Java (B) Go (C) Python (D) Ruby

15. Ansible is described as agentless because
(A) It needs no agent on managed nodes (B) It has no control node (C) It needs no inventory (D) It works only on localhost

16. Ansible playbooks are written in
(A) XML (B) YAML (C) HCL (D) INI only

17. Idempotence in Ansible means
(A) Tasks run in random order (B) Re-running gives the same desired state without extra changes (C) Playbooks run only once (D) Modules are deleted after run

18. The Ansible inventory can be written in
(A) INI or YAML (B) Only XML (C) Only JSON (D) Only binary

19. A task in Ansible calls
(A) Many playbooks (B) A single module (C) A collection (D) A control node

20. Plugins that decide how Ansible connects to a managed node are
(A) Filter plugins (B) Callback plugins (C) Connection plugins (D) Lookup plugins

21. Callback plugins control
(A) Inventory format (B) What is displayed in the console (C) SSH keys (D) Module download

22. A collection can contain
(A) Only modules (B) Playbooks, roles, modules and plugins (C) Only plugins (D) Only inventory

23. Which tool installs collections?
(A) ansible-vault (B) ansible-galaxy (C) ansible-doc (D) ansible-console

24. FQCN of a module looks like
(A) module (B) collection/module (C) namespace.collection.module (D) module@collection

25. Execution Environments are
(A) Ansible control nodes packaged as container images (B) Cloud regions (C) Terraform workspaces (D) YAML linters

26. Best combination of tools per your PPT
(A) Ansible to provision, Terraform to deploy apps (B) Terraform to provision, Ansible to install/deploy apps (C) Only Chef (D) Only Puppet

27. Modules run on the ____, plugins run on the ____
(A) control node, managed node (B) managed node, control node (C) both on managed node (D) both on cloud

28. A handler in a playbook runs
(A) Always first (B) Only when notified by a changed task (C) Never (D) Only at the end of year

29. Which statement about immutable infrastructure is correct?
(A) Servers are modified in place (B) Servers are replaced rather than changed (C) It is not used with cloud (D) It needs manual SSH

30. Which edition of Ansible is a subscription product with enterprise support?
(A) Community Ansible (B) AWX (C) Red Hat Ansible Automation Platform (D) ansible-core

Answer key

1 - B	**2** - C	**3** - B	**4** - C	**5** - C	**6** - B
7 - A	**8** - C	**9** - B	**10** - B	**11** - B	**12** - D
13 - B	**14** - C	**15** - A	**16** - B	**17** - B	**18** - A
19 - B	**20** - C	**21** - B	**22** - B	**23** - B	**24** - C
25 - A	**26** - B	**27** - B	**28** - B	**29** - B	**30** - C

UNIT 4 - DOCKER AND KUBERNETES

Syllabus: Docker overview | Docker architecture | Docker images and containers | Docker Swarm | Kubernetes overview | Kubernetes cluster architecture | Design of Pods.

How this unit maps to the exam

- 10 MCQs: Docker components, commands, network drivers, storage types, K8s components, kubectl commands, pod/service/deployment.
- 15-mark scenario: "application works on dev but not on prod", "deploy a scalable app", "design pods for ...", "compare Docker Swarm and Kubernetes".
- Docker Swarm is in the syllabus but not in your PPT - a full section is given here (red box).

4.1 Docker Overview

Docker is an open-source **containerisation platform** (a set of Platform-as-a-Service products) that packs an application and all its dependencies (libraries, binaries, config) into a standardised unit called a **container**, which runs the same on any machine that has Docker.

Why containers? (the classic scenario)

A developer's code runs on the dev server (Python 2.x) but fails on the test/production server (Python 3.x) because libraries are missing - "works on my machine". Packaging the app with its exact dependencies in a container removes the environment difference.

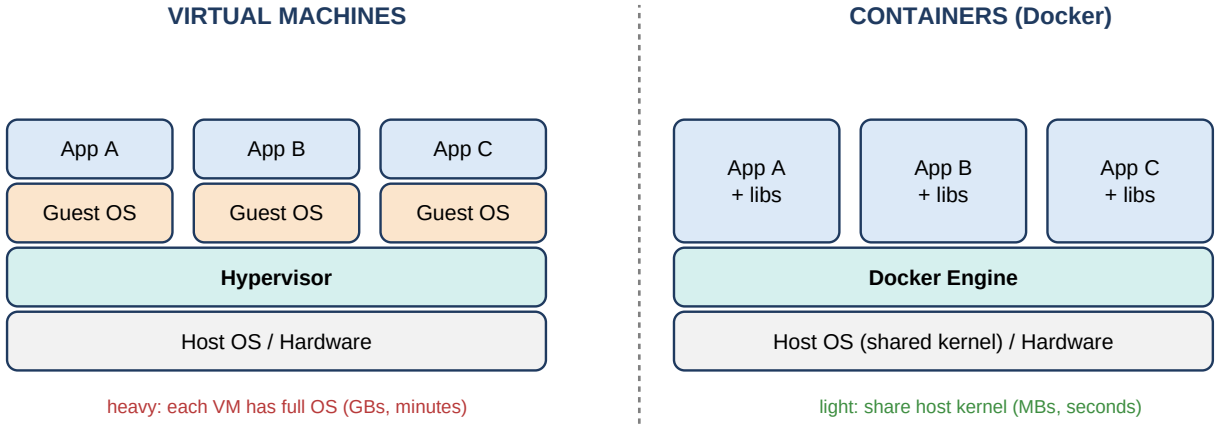


Fig 4.1 Virtual machines vs containers

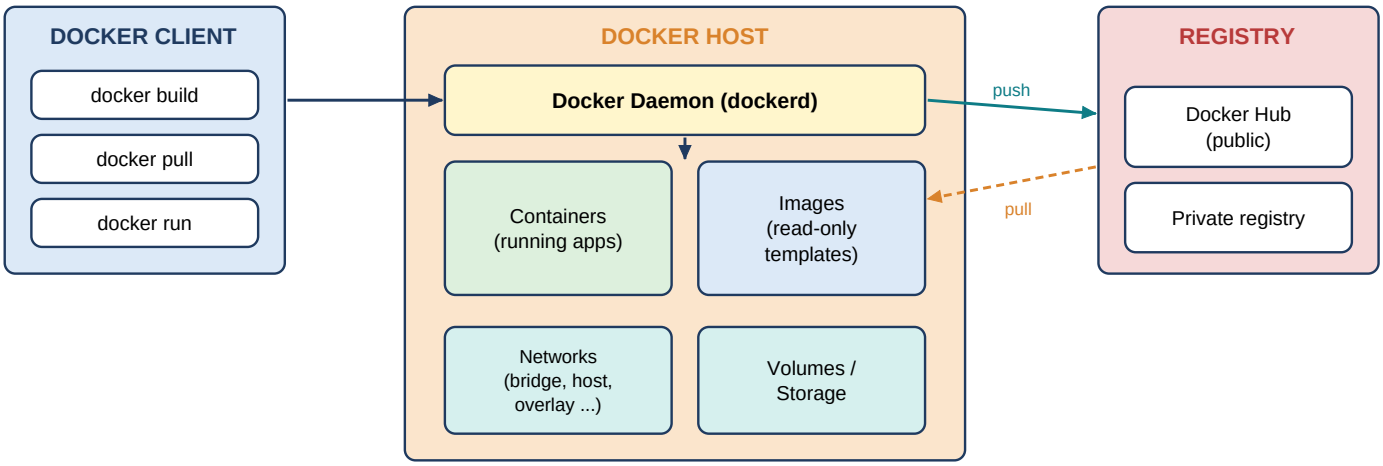
Parameter	Virtual machine	Container
Isolation	Full guest OS per VM (hypervisor)	Process-level isolation; share the host OS kernel
Size	GBs	MBs
Start-up time	Minutes	Seconds or less
Resource use	High - each VM reserves RAM/CPU for an OS	Low - more containers per host
Portability	Moderate	Very high (image runs anywhere with Docker)
Use	Run different OSes, strong isolation	Microservices, CI/CD, scaling apps

Advantages of Docker / containers (PPT)

- **Portability** - one package for dev, test and production.
- **Light weight** - smaller footprint than VMs, fast start.
- **Fast delivery and scalable** - suited to continuous deployment and testing.
- **Isolation** - one container crashing does not affect others.
- **Resource optimisation** - utilise hardware fully; lower cost.
- Works with orchestration platforms (Kubernetes, OpenShift, Azure AKS, Amazon EKS, Google GKE).

4.2 Docker Architecture

Docker uses a **client-server architecture**. The Docker client talks to the Docker daemon, which builds, runs and distributes containers. Client and daemon may be on the same system or connected remotely; they communicate using a **REST API over a UNIX socket or a network**.



Client <-> daemon: REST API over UNIX socket or network

Fig 4.2 Docker architecture - client, host (daemon + objects) and registry

Component	Role (PPT)
Docker client	The way users interact with Docker. "docker" commands use the Docker API; one client can talk to multiple daemons. Terminal sends instruction -> daemon processes it.
Docker daemon (dockerd)	Listens for API requests and manages Docker objects - images, containers, networks, volumes; can communicate with other daemons.
Docker host	Machine that runs the daemon and hosts images, containers, networks and storage; can run many containers.
Docker registry	Stores Docker images. **Docker Hub** is the public registry (free for public images, paid for private); you can also run a private registry. `docker pull / docker run` fetch from the registry; `docker push` uploads.
Docker objects	Images, containers, networks, storage (volumes) - created and used by the daemon.

What happens when you run "docker run nginx"?

- Client sends the command to the daemon via REST API.
- Daemon looks for the image locally; if absent, **pulls it from the registry**.
- Daemon creates a container from the image (adds a writable layer), sets up networking and storage.
- Daemon starts the container process; output goes back to the client.

4.3 Docker Images and Containers



Fig 4.3 From Dockerfile to a running container (docker push/pull move images to/from a registry)

Docker image

- A **read-only template** containing instructions for creating a container (code, runtime, libraries, config).
- Built from a **Dockerfile**; made of stacked **layers** - cached and shared, so rebuilds and pulls are fast.
- Used to store and ship applications; stored in registries; enables collaboration between developers.

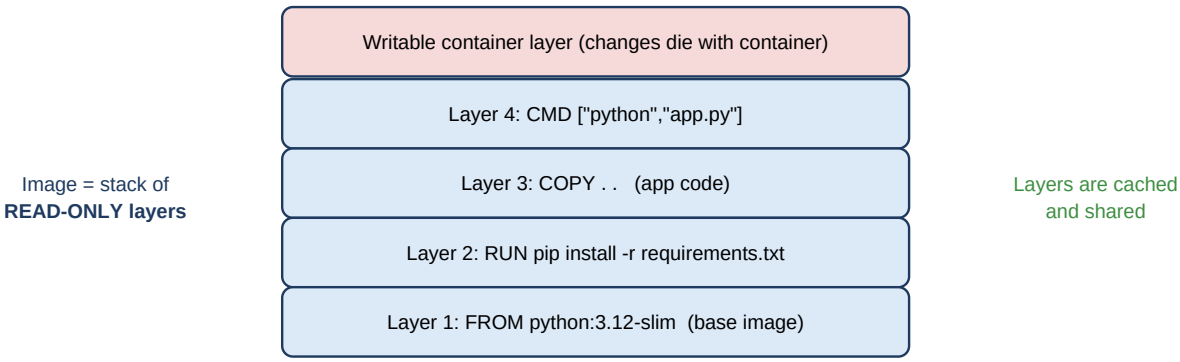


Fig 4.4 Docker image layers and the container writable layer

```
# Dockerfile
# base image
FROM python:3.12-slim
# working directory inside the container
WORKDIR /app
# copy dependency list, then install it (cached layer)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# copy application code
COPY . .
# document the port the app listens on
EXPOSE 5000
# default command when the container starts
CMD ["python", "app.py"]
```

Docker container

- A **runnable instance of an image**; created from images with a writable layer on top.
- Managed with the Docker API/CLI - start, stop, delete, move.
- Can access only the resources defined in the image unless extra access is given (ports, volumes).
- Lifecycle: created -> running -> paused -> stopped -> removed.

Image	Container
Read-only template (blueprint)	Running (or stopped) instance of an image
Built once with docker build	Created many times with docker run
Stored in registry / local cache	Lives on the Docker host; has writable layer; data lost on removal unless volume used
Class (analogy)	Object (analogy)

Docker storage (PPT)

Data written in a container's writable layer is lost when the container is removed and needs a storage driver (manages images and containers on the host). For persistence use:

Type	Description
Data volumes	Directories/files on the host filesystem mounted into the container; managed by Docker; survive container deletion.
Volume container	Container whose volumes are shared with other containers - independent life cycle, easy to share and back up data.
Directory mounts (bind mounts)	A specific host directory mounted into the container.
Storage plugins	Volume plugins connect containers to external storage such as Amazon EBS.

Docker networking (PPT)

Docker networking gives containers isolation; a container can be connected to many networks.

Driver	Use
bridge	**Default** driver - containers on the same Docker host communicate.

Driver	Use
host	No isolation between container and host - shares host network.
overlay	Connects containers across multiple hosts; enables **swarm services** to communicate.
none	Disables all networking.
macvlan	Assigns a MAC address to the container so it appears as a physical device.

Docker CLI cheat-sheet (from PPT slides)

Command	Meaning
docker build -t myapp:1.0 .	Build image from Dockerfile
docker images	List images on the host
docker run -d -p 8080:80 --name web nginx	Run a container (detached, host port 8080 -> container 80)
docker ps / docker ps -a	List running / all containers
docker stop web / docker rm web	Stop / remove a container
docker rmi nginx	Remove an image
docker pull / docker push	Download from / upload to a registry
docker logs web / docker exec -it web bash	View logs / open a shell in a running container
docker volume ls / create / inspect / prune / rm	Manage volumes
docker info / docker -v	Server details (containers, images, storage driver) / version
docker system df / prune	Disk usage / remove unused data

4.4 Docker Swarm

NOT IN YOUR PPT - ADD FROM SYLLABUS

- Your Unit-4 PPT mentions Swarm only once (overlay network "enables swarm services"). Learn this section from the syllabus.

Docker Swarm is Docker's native **container orchestration** / clustering tool. **Swarm mode** turns a group of Docker hosts into a single virtual Docker engine so you can deploy, scale and manage containers across many machines with high availability.

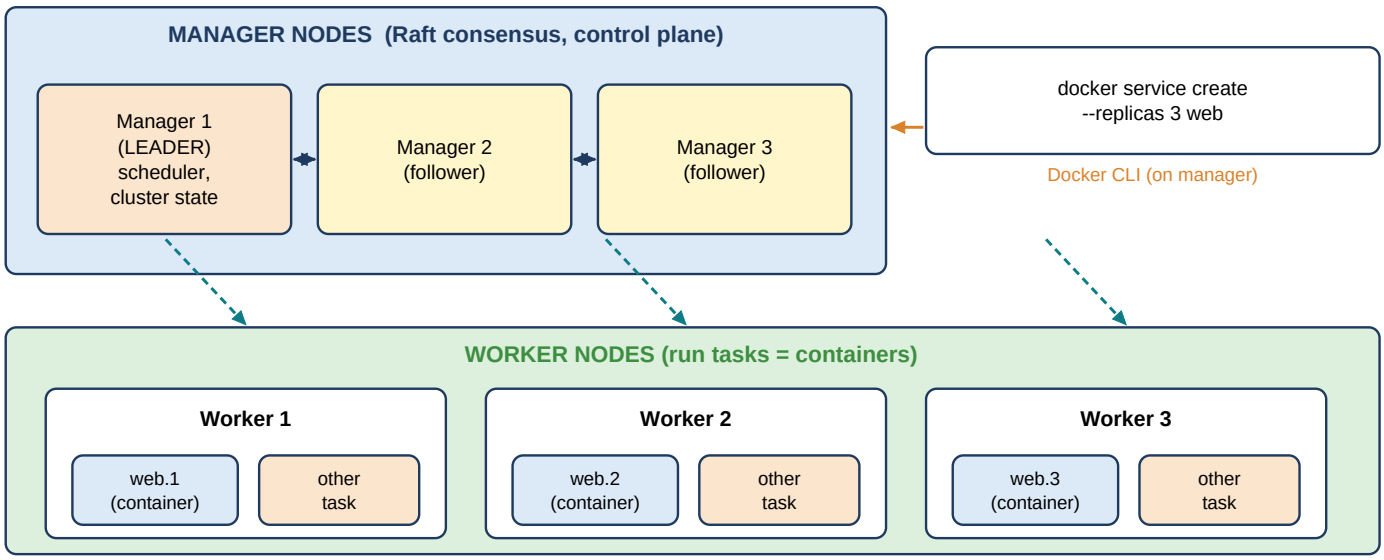


Fig 4.5 Docker Swarm cluster - dashed arrows = task assignment; overlay network + routing mesh connect all nodes

Term	Meaning
Node	A Docker engine participating in the swarm. **Manager** nodes and **worker** nodes.
Manager node	Maintains cluster state with **Raft consensus**, schedules services, serves the API. One leader elected; use an odd number (3 or 5) for fault tolerance.

Term	Meaning
Worker node	Receives and runs **tasks** assigned by managers.
Service	Desired state definition (image, replicas, ports, update policy). **Replicated** service = N copies; **global** service = one task on every node.
Task	A single running container scheduled on a node - the atomic unit of scheduling.
Overlay network	Multi-host network so containers on different nodes talk to each other.
Routing mesh	Published port is reachable on every node; request is routed to a node that runs a task.
Stack	A group of services deployed from a Compose file (docker stack deploy).
Secrets	Encrypted sensitive data delivered only to services that need them.

Swarm in action - commands

```
docker swarm init --advertise-addr 192.168.1.10      # make this node a manager
docker swarm join-token worker                        # get the join command for workers
docker swarm join --token <TOKEN> 192.168.1.10:2377   # run on each worker
docker node ls                                        # list nodes (on manager)

docker service create --name web --replicas 3 -p 80:80 nginx    # create replicated service
docker service ls ; docker service ps web                       # inspect
docker service scale web=5                                      # scale out
docker service update --image nginx:1.25 web                    # rolling update
docker service rollback web                                     # undo the update
docker stack deploy -c docker-compose.yml myapp                 # deploy a stack
docker swarm leave                                              # leave the swarm
```

- **Self-healing:** if a node or container fails, the manager reschedules the task elsewhere to keep the desired replica count.
- **Rolling updates** with delay and parallelism; **load balancing** built in (ingress + internal DNS).

Docker Swarm vs Kubernetes

Parameter	Docker Swarm	Kubernetes
Setup	Very easy (docker swarm init)	More complex (kubeadm, managed services)
Unit of deployment	Service / task (container)	Pod (one or more containers)
Scaling	Manual (service scale); limited autoscaling	Rich - Horizontal Pod Autoscaler, cluster autoscaler
Load balancing	Built-in routing mesh	Services, Ingress, external load balancers
Self-healing and rollbacks	Yes, basic	Yes, advanced (probes, ReplicaSets, rollout history)
Ecosystem / market	Smaller	Industry standard, huge ecosystem, cloud-managed (EKS, AKS, GKE)
Best for	Small/medium teams, quick Docker-native clustering	Large, complex, production-grade workloads

4.5 Kubernetes Overview

Kubernetes (K8s, "kube"; Greek for "helmsman") is an open-source **container orchestration platform**, developed by Google, for automating **deployment, scaling and operations** of containerised applications across a cluster of nodes. It supports container runtimes such as containerd, CRI-O and Kata containers.

- **Highly resilient infrastructure:** zero-downtime deployment, automatic rollback, scaling and **self-healing** (auto-placement, auto-restart, auto-replication, scaling on metrics).
- **Objective:** hide the complexity of managing a fleet of containers by providing REST APIs.
- **Portable:** runs on AWS, Azure, OpenStack, Google Cloud, bare metal.
- **Declarative:** you give desired state (YAML); controllers keep making the actual state match it (autonomous processes reacting to events from the API server).
- **Hosted services:** EKS (AWS), AKS (Azure), GKE (Google), Alibaba, DigitalOcean, IBM, Oracle.
- **Local clusters:** Docker Desktop, Minikube, Kind, k3s.

How it works (PPT): there is a master node and many worker nodes; each worker handles many pods. Pods are a bunch of containers working as a unit. You give pod definitions and the replica count to the master; from there Kubernetes takes control and deploys the pods to the worker nodes.

Docker vs Kubernetes - not competitors

Docker	Kubernetes
Builds and runs containers on one host	Orchestrates containers across many hosts
Unit: container	Unit: pod
docker CLI	kubectl CLI
Swarm gives basic clustering	Self-healing, autoscaling, rolling updates, service discovery

4.6 Kubernetes Cluster Architecture

Kubernetes follows a **client-server (master-worker) architecture**: a control plane (master) that makes decisions and worker nodes that run the application pods. A multi-master setup gives high availability.

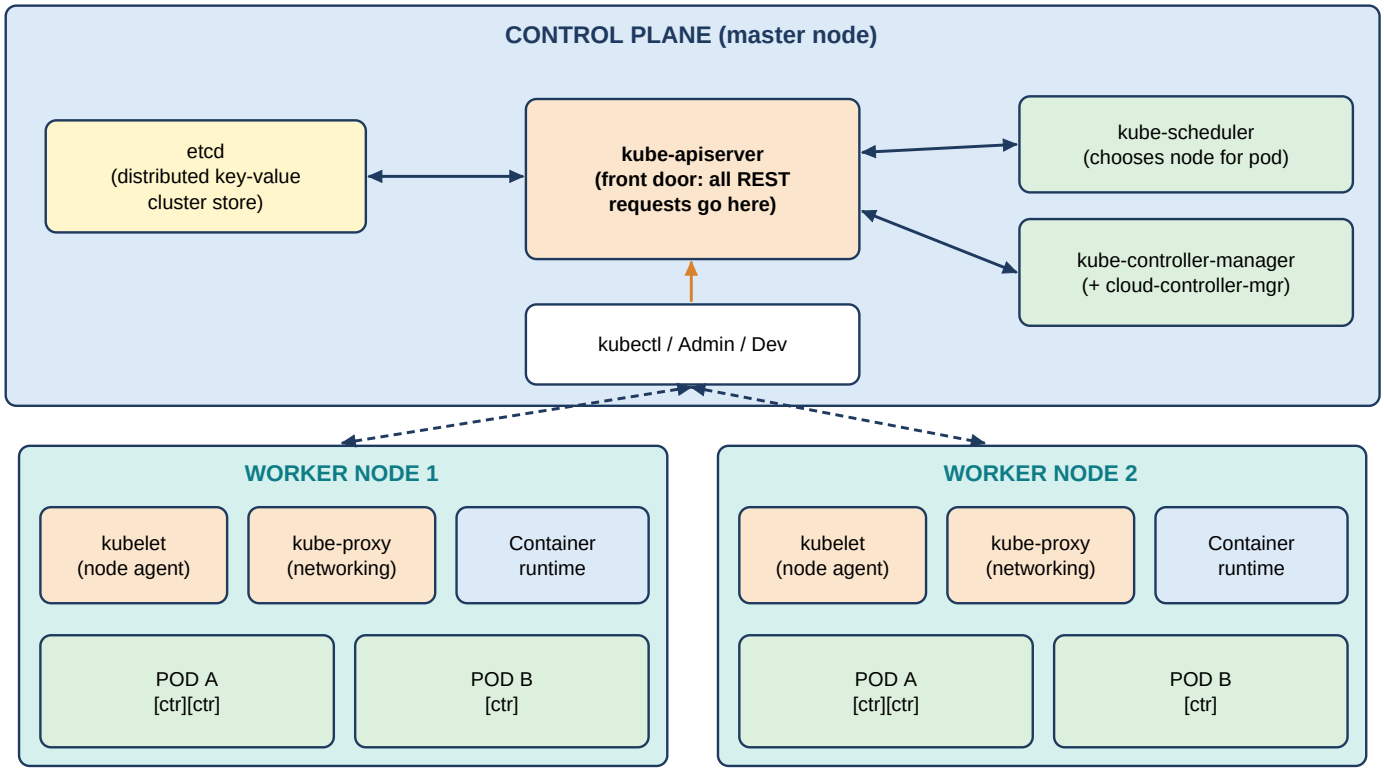


Fig 4.6 Kubernetes cluster architecture (control plane + worker nodes)

Control plane (master) components

Component	Function
etcd	Distributed, consistent key-value store holding all cluster data (pods, state, namespaces, API objects, service discovery details). Only the API server talks to it.
kube-apiserver	Central management entity and **front end** of the cluster; receives all REST requests (from kubectl, other components) and validates/stores objects.
kube-scheduler	Watches for unscheduled pods and assigns each to the best-fit node based on resource needs (e.g. 1 GB RAM, 2 CPU), policies, taints/labels.
kube-controller-manager	Runs controllers: replication controller keeps replica count, endpoints controller populates endpoints, node controller etc. - regulates shared state.
cloud-controller-manager	Handles controllers that depend on the cloud provider (node termination checks, routes, load balancers, volumes).

Worker node components

Component	Function
kubelet	Main agent on a node; takes pod specs (via API server), makes sure containers are running and healthy, reports node/pod status to master.
kube-proxy	Network proxy on each node; maintains network rules (iptables), handles pod-to-service routing and exposes services; forwards requests to correct pods.
Container runtime	Software that runs containers - containerd, CRI-O (Docker Engine via cri-dockerd).
Pods	The workloads that run on the node.

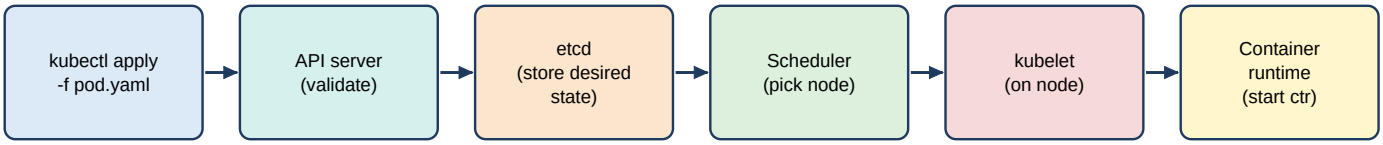


Fig 4.7 What happens when you create a pod

- **kubectl** is the command-line client; every command is converted to an API call to kube-apiserver.
- **Slide Q&A:** if the control plane goes down, running services keep working as long as nothing changes (kubelet and kube-proxy rules on nodes continue; e.g. LB-exposed service still works), but no new scheduling/changes are possible.

Kubernetes objects / resources (PPT table)

Group	Resource	Description
Workloads	Pod	Basic deployable unit; one or more co-located containers
	ReplicaSet / ReplicationController	Keeps N pod replicas running (RC = older equivalent)
	Deployment	Declares desired state; manages ReplicaSets; rolling update/rollback
	Job / CronJob	Run-to-completion task / scheduled task
	DaemonSet	One pod per node
	StatefulSet	Pods with stable identity and storage
Services	Service (svc)	Stable IP/port for a set of pods
	Endpoints	Which pods are exposed through a service
	Ingress	Exposes services to external clients through one externally reachable IP
Config	ConfigMap / Secret	Non-sensitive / sensitive configuration
Storage	PersistentVolume (+Claim)	Persistent storage mounted into pods
Scaling	HorizontalPodAutoscaler	Scale replicas on CPU or other metric
Cluster	Namespace, Node	Logical group of resources / worker node
Security	ServiceAccount, Role, RoleBinding, ClusterRole	RBAC - who can do what

4.7 Design of Pods

A **Pod** is the smallest deployable unit in Kubernetes: one or more **tightly coupled containers** controlled as a single application. A pod encapsulates the application containers, storage resources, a **unique network ID (IP)** and options on how to run the containers. Pods are **ephemeral** - Kubernetes does not guarantee a given pod stays alive - so controllers (Deployment/ReplicaSet) and Services are used.

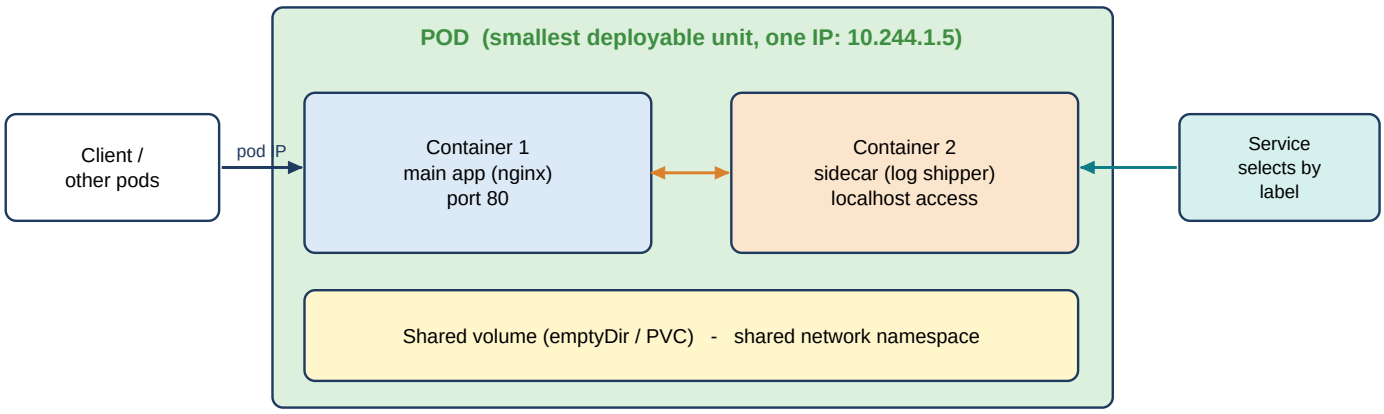


Fig 4.8 Multi-container pod - shared network and storage

Design principles

- **One main process per container; group containers in a pod only if they must share lifecycle, network and storage** (e.g. app + log shipper).
- **Shared network namespace:** every pod has its own unique IP shared by all its containers; containers talk over `localhost` and must use different ports.
- **Shared storage:** volumes declared in the pod are mountable by all its containers.
- **Scale by pods, not containers:** to scale add pod replicas (via Deployment).
- **Pods are disposable:** never rely on pod IP; use Services; store state outside (PersistentVolume).
- **Labels and selectors** link pods to Services and controllers.

Kubernetes networking rules (PPT)

- Every pod has a unique IP, routable from all other pods; all containers in a pod can communicate with each other.
- All pods can talk to all other pods **without NAT**; all nodes can talk to all pods (and vice-versa) without NAT.
- The IP a pod sees itself as is the same IP others see it as.

Multi-container pod patterns

Pattern	Idea	Example
Sidecar	Helper container extends/enhances the main one	Log shipper, config reloader, service-mesh proxy
Ambassador	Proxy for outside services	Local proxy to a database cluster
Adapter	Standardises/transforms output	Convert app metrics to Prometheus format
Init container	Runs to completion BEFORE app containers start	Wait for DB, download config, run migrations

Pod YAML (from PPT) and a two-container pod

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web-with-logger
  labels: { app: web }
spec:
  volumes:
  - name: logs
    emptyDir: {}                      # shared scratch volume
  initContainers:
  - name: wait
    image: busybox
    command: ["sh","-c","sleep 3"]
  containers:
  - name: web                         # main container
    image: nginx:1.25
    ports: [{ containerPort: 80 }]
    resources:
      requests: { cpu: "100m", memory: "64Mi" }
      limits:   { cpu: "250m", memory: "128Mi" }
    readinessProbe: { httpGet: { path: /, port: 80 }, initialDelaySeconds: 3 }
    livenessProbe:  { httpGet: { path: /, port: 80 }, periodSeconds: 10 }
    volumeMounts: [{ name: logs, mountPath: /var/log/nginx }]
  - name: log-shipper                 # sidecar
    image: busybox
    command: ["sh","-c","tail -F /logs/access.log"]
    volumeMounts: [{ name: logs, mountPath: /logs }]
```

Pod design element	Why it matters
Phases	Pending -> Running -> Succeeded / Failed (Unknown if node lost)
restartPolicy	Always (default), OnFailure, Never
Liveness probe	Restarts the container if it is unhealthy
Readiness probe	Removes pod from Service endpoints until ready
Resource requests / limits	Scheduler placement and protection against noisy neighbours
Labels	Selectors for Service/Deployment (e.g. app: web)
Namespace	Logical separation (per team/tenant)

Controllers and Services around pods

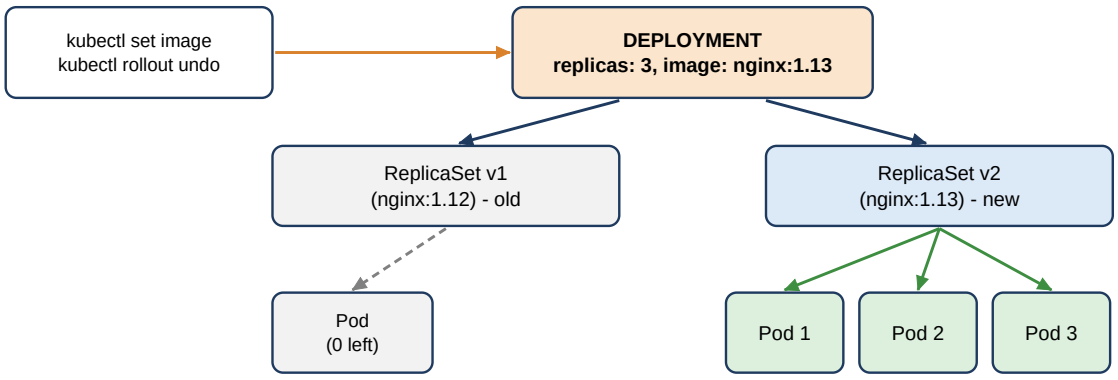


Fig 4.9 Deployment manages ReplicaSets, which manage Pods (rolling update and rollback)

```
apiVersion: apps/v1                     # Deployment (PPT)
kind: Deployment
metadata:
  name: my-nginx
spec:
  selector:
    matchLabels: { run: my-nginx }
  replicas: 2
  template:                              # pod template
    metadata:
      labels: { run: my-nginx }
    spec:
      containers:
      - name: my-nginx
        image: nginx
        ports: [{ containerPort: 80 }]
---
apiVersion: v1                          # Service (PPT)
kind: Service
metadata:
  name: my-nginx
  labels: { run: my-nginx }
spec:
  ports: [{ port: 80, protocol: TCP }]
  selector: { run: my-nginx }
```

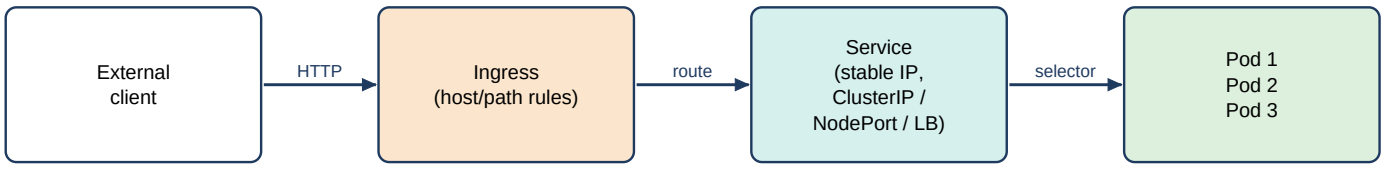


Fig 4.10 Exposing pods: client -> Ingress -> Service -> Pods

Service type	Behaviour
ClusterIP (default)	Internal virtual IP reachable only inside the cluster
NodePort	Opens a static port on every node (30000-32767)
LoadBalancer	Provisions a cloud load balancer (MetalLB on bare metal)
Ingress	L7 HTTP/HTTPS routing (host/path rules) to many Services via an Ingress controller
Headless (ClusterIP: None)	No virtual IP; DNS returns pod IPs directly - used for stickiness / StatefulSets

kubectl commands and the rolling-update exercise (PPT)

Task	Command
Create deployment, 2 pods	kubectl create deployment nginx --image=nginx:1.12.2 --replicas=2 (PPT: kubectl run nginx --image=nginx:1.12.2 --replicas=2)
Scale	kubectl scale --replicas=5 deployment nginx
Upgrade image	kubectl set image deployment nginx nginx=nginx:1.13.8
Check upgrade status	kubectl rollout status deployment nginx
History / rollback	kubectl rollout history deployment nginx / kubectl rollout undo deployment nginx
Expose service	kubectl expose deployments nginx --port=80 --type=LoadBalancer ; kubectl get svc
Autoscale (HPA)	kubectl autoscale deployment ghost --min=1 --max=4 --cpu-percent=10 ; kubectl get hpa
Inspect	kubectl get pods -o wide / describe pod X / logs X / exec -it X -- bash / get nodes / cluster-info
Apply a manifest	kubectl create -f nginx.yaml or kubectl apply -f nginx.yaml

- **Deployment strategies:** RollingUpdate (default - gradually replace pods, zero downtime), Recreate (kill all then create), and release patterns blue/green and canary.
- **HPA flow (PPT slide):** metrics-server collects metrics periodically -> threshold met? -> HPA asks the Deployment to change replicas -> new pods deployed.
- **Three ways to expose services:** NodePort, external LoadBalancer, Ingress (controller + resource).

- **Security best practices (PPT):** scan images, use authorised images only, namespaces, network policies, RBAC, limit SSH (use kubectl exec), keep secrets in Secrets/Vault not plain text, run as non-root.

4.8 Scenario Questions with Model Answers (15 marks each)

EXAM TIP

- Use the same structure: Definition -> Diagram -> Explanation in steps -> Commands/YAML -> Benefits -> Conclusion.
- In scenarios say the reason for the technology: "containers because the environment must be identical", "Kubernetes because auto-scaling and self-healing are needed".

Q1 (Either) A developer's app works on the dev laptop but crashes on the test server because of missing libraries. How does Docker solve this? Explain Docker architecture and the container workflow.

- **Problem:** environment inconsistency (library/version mismatch). **Container** bundles app + dependencies (Fig 4.1 vs VM).
- **Architecture (Fig 4.2):** client, daemon, host, registry, objects; REST API over UNIX socket/network.
- **Workflow (Fig 4.3):** write Dockerfile -> docker build -> image -> docker push to registry -> on test server docker pull + docker run. Show the Dockerfile and 4 commands.
- **Data and network:** volumes for persistence; bridge network and -p port mapping.
- **Benefits:** portability, light weight, fast start, isolation, resource efficiency. Conclusion.

Q1 (Or) Differentiate image and container; explain Docker storage and networking types.

- Image vs container table + layers (Fig 4.4); four storage types; five network drivers with use of each; one-line example for each.

Q2 (Either) An e-commerce site must stay available when one server fails and must scale to 5 replicas. Design a solution with Docker Swarm.

- Define Swarm; draw Fig 4.5 (3 managers with Raft, workers, service, tasks, overlay + routing mesh).
- **Setup:** docker swarm init on manager 1, join workers/managers with tokens; verify docker node ls.
- **Deploy:** docker service create --name web --replicas 3 -p 80:80 nginx; scale to 5; rolling update with docker service update --image; rollback if needed.
- **Failure scenario:** a worker dies -> manager notices, reschedules its tasks on healthy nodes to keep desired state; manager leader failure -> Raft elects a new leader (needs majority, hence 3 or 5 managers).
- Benefits (HA, load balancing, simple) and limits vs Kubernetes.

Q2 (Or) Compare Docker Swarm and Kubernetes.

- Use the comparison table in 4.4 and add one scenario for each (small team quick cluster vs large autoscaling workloads).

Q3 (Either) Explain Kubernetes cluster architecture with a neat diagram and describe what happens internally when "kubectl apply -f pod.yaml" is executed.

- Draw Fig 4.6; describe each control plane and worker component in one or two lines (tables in 4.6).
- **Flow (Fig 4.7):** kubectl -> API server authenticates/validates -> writes desired state to etcd -> scheduler sees unscheduled pod and binds it to a node -> kubelet on that node sees the assignment -> instructs container runtime to pull image and start containers -> kubelet reports status back; controllers keep the state.
- Mention declarative model and self-healing; kube-proxy sets service rules. Conclusion.

Q3 (Or) What is Kubernetes? Explain its features, objectives and the main objects (Pod, Service, Deployment, ReplicaSet, ConfigMap, Secret).

- Overview section 4.5 + objects table 4.6 + the three YAMLs.

Q4 (Either) A web application needs an nginx container and a log-shipping container that share logs. Design the pod, and expose it to users with zero-downtime updates.

- **Pod design:** multi-container (sidecar) pod; shared emptyDir volume and localhost network (Fig 4.8); init container if needed; probes and resource limits (YAML in 4.7).
- **Why a pod:** containers need same lifecycle, IP and storage. Pods are ephemeral -> manage with Deployment (replicas: 3).
- **Expose:** Service (selector by label) type LoadBalancer or Ingress for host/path routing (Fig 4.10).
- **Zero-downtime update:** kubectl set image ... (RollingUpdate), watch with rollout status, rollback with rollout undo; readiness probe prevents traffic to unready pods (Fig 4.9).
- **Scaling:** HPA on CPU. Conclusion.

Q4 (Or) Explain pod lifecycle, pod networking and the multi-container pod patterns.

- Phases, restartPolicy, probes; networking rules (unique IP, no NAT, localhost); sidecar/ambassador/adapter/init with examples.

4.9 MCQ Practice - Unit 4 (35 questions)

Attempt without looking, then check the key at the end of this section. Each MCQ in CT2 is one-line recall - read the "Key points" boxes before attempting.

1. Docker is a
(A) Hypervisor (B) Containerisation platform (C) Programming language (D) Database

2. Containers differ from VMs because containers
(A) Include a full guest OS (B) Share the host OS kernel (C) Need a hypervisor (D) Are always larger

3. The Docker client and daemon communicate using
(A) SMTP (B) REST API over UNIX socket/network (C) FTP (D) SNMP

4. Which component manages images, containers, networks and volumes?
(A) Docker client (B) Docker daemon (C) Docker Hub (D) Dockerfile

5. Docker Hub is a
(A) Public registry (B) Container runtime (C) Network driver (D) Orchestrator

6. A Docker image is
(A) A running process (B) A read-only template (C) A volume (D) A network

7. Which command builds an image from a Dockerfile?
(A) docker run (B) docker build (C) docker pull (D) docker exec

8. Which command lists running containers?
(A) docker images (B) docker ps (C) docker rmi (D) docker info

9. Which command removes an image?
(A) docker rm (B) docker rmi (C) docker stop (D) docker prune

10. Default Docker network driver is
(A) host (B) none (C) bridge (D) overlay

11. Which network driver disables all networking?
(A) none (B) host (C) macvlan (D) overlay

12. Which driver connects swarm services across hosts?
(A) bridge (B) overlay (C) none (D) host

13. macvlan network driver
(A) Disables networking (B) Assigns a MAC address to containers (C) Shares host network (D) Is the default

14. Docker volumes are used to
(A) Build images (B) Persist container data (C) Pull images (D) Scale services

15. Docker Swarm manager nodes use ____ for consensus
(A) Paxos (B) Raft (C) Gossip only (D) BGP

16. Command to scale a swarm service to 5 replicas
(A) docker service scale web=5 (B) docker scale web 5 (C) docker swarm scale 5 (D) docker run --scale 5

17. A swarm "global" service runs
(A) One task on every node (B) Only on manager (C) Only one replica (D) Only on leader

18. Kubernetes was originally developed by
(A) Microsoft (B) Google (C) Red Hat (D) Docker

19. Smallest deployable unit in Kubernetes
(A) Container (B) Pod (C) Node (D) Service

20. Which component stores the cluster data in K8s?
(A) kubelet (B) etcd (C) kube-proxy (D) scheduler

21. Front end / central management entity of the control plane
(A) kube-apiserver (B) kubelet (C) etcd (D) kube-proxy

22. Which component assigns pods to nodes?
(A) kube-scheduler (B) kubelet (C) kube-proxy (D) etcd

23. Agent that runs on each worker node and keeps pods healthy
(A) kubelet (B) scheduler (C) apiserver (D) etcd

24. Which component handles node-level networking rules for Services?
(A) kube-proxy (B) etcd (C) scheduler (D) cloud-controller

25. Command-line tool for Kubernetes
(A) kubectl (B) kubeadmin (C) k8sctl (D) docker-k8s

26. Which object provides a stable IP for a set of pods?
(A) Deployment (B) Service (C) ConfigMap (D) Namespace

27. Which object manages rolling updates and rollbacks?
(A) Deployment (B) Pod (C) Node (D) Secret

28. Command to roll back a deployment
(A) kubectl rollout undo deployment nginx (B) kubectl delete rollout (C) kubectl restore nginx (D) kubectl revert pod

29. Containers inside the same pod communicate using
(A) Public internet only (B) localhost (C) Pod must be on another node (D) Docker bridge only

30. Sensitive data in Kubernetes should be stored in
(A) ConfigMap (B) Secret (C) Pod label (D) Namespace

31. Which Service type opens a static port on every node?
(A) ClusterIP (B) NodePort (C) ExternalName only (D) Headless

32. HorizontalPodAutoscaler changes
(A) Node CPU (B) Number of pod replicas (C) Image size (D) Namespace

33. A DaemonSet runs
(A) One pod per node (B) Only one pod in cluster (C) Jobs on schedule (D) Only on master

34. Which statement about pods is true?
(A) Each pod has its own unique IP (B) Pods share one IP cluster-wide (C) Pods cannot have volumes (D) Pods never restart

35. Init containers
(A) Run before app containers and must finish (B) Run after pod deletion (C) Are optional sidecars that run forever (D) Replace kubelet

Answer key

1 - B	**2** - B	**3** - B	**4** - B	**5** - A	**6** - B
7 - B	**8** - B	**9** - B	**10** - C	**11** - A	**12** - B
13 - B	**14** - B	**15** - B	**16** - A	**17** - A	**18** - B
19 - B	**20** - B	**21** - A	**22** - A	**23** - A	**24** - A
25 - A	**26** - B	**27** - A	**28** - A	**29** - B	**30** - B
31 - B	**32** - B	**33** - A	**34** - A	**35** - A	

ONE-PAGE QUICK REVISION SHEETS

Unit 3 - rapid recall

Term	One-liner
IaC	Managing infrastructure through machine-readable code: version-controlled, automated, repeatable.
Push vs pull	Push = central server sends config (Ansible). Pull = nodes fetch config (Puppet/Chef agents).
Declarative / procedural	Declare end result (Terraform) / give step-by-step instructions.
Mutable / immutable	Change servers in place / replace servers instead of changing.
Terraform	HashiCorp, HCL, declarative, multi-cloud, providers + state file.
Terraform commands	init (plugins) -> validate (syntax) -> plan (preview: add/modify/delete) -> apply (type yes) -> destroy.
Terraform files	main.tf, variables.tf, outputs.tf, terraform.tfvars, terraform.tfstate.
Variable types	string, number, bool, list, map, set, object; use as var.name; override with -var, tfvars, TF_VAR_.
Ansible	Red Hat, Python, YAML, agentless (SSH), idempotent, push model.
Inventory	INI or YAML; groups, host vars; static or dynamic.
Playbook hierarchy	Playbook > Play (hosts) > Tasks > Module; roles and handlers.
Module vs plugin	Module runs on managed node to do the task; plugin runs on control node to extend Ansible.
Plugin types	connection, filter, callback, lookup, inventory, become, strategy.
Collection	namespace.collection bundling modules/plugins/roles/playbooks; ansible-galaxy collection install; call by FQCN.
EE	Control node packaged as container image (ansible-builder, ansible-navigator).
Tool pairing	Terraform provisions; Ansible configures/deploys.

Unit 4 - rapid recall

Term	One-liner
Docker	Containerisation platform; packs app + dependencies into a container.
Architecture	Client -> (REST API) -> daemon on host (images, containers, networks, volumes) <-> registry (Docker Hub).
Image / container	Read-only template / running instance with writable layer.
Dockerfile instructions	FROM, WORKDIR, COPY, RUN, EXPOSE, CMD.
Storage	Data volumes, volume container, directory (bind) mounts, storage plugins.
Network drivers	bridge (default), host, overlay (swarm), none, macvlan.
Swarm	Managers (Raft, odd number) + workers; service -> tasks; overlay + routing mesh; docker swarm init / service create / scale / update.
Kubernetes	Google, orchestration: self-healing, scaling, rolling updates, declarative YAML.
Control plane	kube-apiserver, etcd, scheduler, controller-manager, cloud-controller-manager.
Worker node	kubelet, kube-proxy, container runtime, pods.
Pod	Smallest unit; shared IP + volumes; ephemeral; sidecar/ambassador/adapter/init patterns.
Deployment	Manages ReplicaSets -> pods; scale, rolling update, rollout undo.
Service types	ClusterIP, NodePort, LoadBalancer; Ingress for HTTP routing; headless for stickiness.
kubectl essentials	get, describe, apply/create -f, scale, set image, rollout status/history/undo, expose, autoscale, logs, exec.

Most-likely 15-mark topics (ranked)

- Terraform: workflow + EC2 example with variables/outputs (diagram + code).

- Ansible: inventory + playbook scenario; agentless architecture; idempotency; handlers.
- IaC: benefits, push vs pull, declarative vs procedural; Terraform vs Ansible.
- Ansible modules, plugins and collections (with FQCN usage in a playbook).
- Docker architecture + image/container workflow + storage/network.
- Docker Swarm cluster design (HA + scaling + rolling update).
- Kubernetes cluster architecture + pod-creation flow.
- Pod design: multi-container, probes, Deployment + Service YAML, rolling update.

KEY POINTS TO REMEMBER

- Always write **desired state**, **declarative**, **idempotent** for Terraform/Ansible and **self-healing, scaling, rolling update** for Kubernetes.
- Docker = build and run containers; Swarm/Kubernetes = orchestrate many containers across many hosts.
- Best of luck for CT2 on 29.10.2026!